Angular Change Detection

Complete Consolidated Guide

August 20, 2026

TOPIC — ANGULAR CHANGE DETECTION

Angular Core | Critical Relevance

Simple Definition

Change Detection is the process Angular runs to keep the DOM in sync with your component's current state - checking template bindings for changes and patching only the parts of the DOM that actually changed.

Everything else - triggers, tree walks, OnPush, Signals - is just how and when that sync happens, not what it is.

Why the DOM Needs to Be Synced With Component State

The DOM shows whatever was last rendered. A component's data (this.count, a signal, etc.) can change independently in JS memory - a click handler runs, an HTTP response arrives - but the browser has no idea that happened. Nobody told it. Without something re-checking and re-writing the affected DOM nodes, the screen would freeze at whatever it looked like on first render while the actual app state moves on underneath it, invisibly.

Why It Isn't Synced Automatically by Default

Plain JavaScript variables carry zero notification mechanism. this.count = 5 is just a memory write - there's no built-in hook that fires "hey, something read count in a template, better go update that DOM node." The DOM and the JS object graph are two separate data structures with no wiring between them unless something builds that wiring.

That's the real fork in how Angular solves it:

```
Zone.js approach:  don't try to know exactly what changed - just
                   re-check EVERYTHING whenever ANYTHING async
                   happens, and patch whatever's different.

Signals approach:  make state changes self-announcing - a signal
                   write directly notifies the exact template
                   location that read it, so there's no need to
                   guess or re-check broadly.
```

Both exist to solve the same root fact: state changing in memory does not, by itself, cause the DOM to change. Something always has to bridge that gap - CD is Angular's bridge.

The Bridge Model — Structural, Not Physical

The DOM is on the browser and is rendered by the browser using the compiled instructions of Angular's template. The component state technically lives in the same JS engine's memory (the JS heap) as the rest of the browser process - so the disconnect is not physical distance, it's structural: the DOM is one data structure (a tree the renderer paints from), the component class is a completely separate data structure (a plain JS object graph), and JavaScript's execution model doesn't link writes to one with reads from the other.

DOM tree <-> (no automatic link) <-> Component state (JS object/signal in memory) CD is the thing that manually walks that gap - reads current state, compares to what's currently painted, writes the difference into the DOM.

CD reads the data from the JS heap (the component state) and writes it to the DOM - meaning it patches the updates, if any, to the DOM node - so the UI is always displayed with the updated content. That is the complete mechanism, distilled: read state from heap, compare to last-painted value, patch DOM node if different.

[redacted]

Full Mental Model

Named analogy: A teacher walking down every row in a classroom, tapping each student's desk and asking "did your answer change since I last asked?" - that's Zone.js-era CD. Signals-based CD is different: it's students raising their hand the instant their answer changes, and the teacher only walks to that desk.

Why It Exists At All

JavaScript objects aren't reactive by default. this.count = 5 doesn't tell the DOM anything. Something has to notice the change and re-run the template to sync {{ count }} with reality. Change detection is Angular's answer to "how do I know when to re-render."

The Core Mechanism — Two Eras

Era 1 - Zone.js (legacy, still default unless zoneless is opted into): Zone.js monkey-patches every async browser API - setTimeout, addEventListener, fetch, Promises. Any async operation completing triggers Angular to run a full CD pass - the "teacher walks every row" model.

```
Async event fires (click, HTTP response, timer)
  -> Zone.js intercepts it
  -> notifies Angular: "something happened, might have changed state"
  -> Angular runs CD from root component downward
```

Era 2 - Zoneless + Signals (the default mental model for this learning track): No monkey-patching. Angular tracks exactly which signals are read inside which template, via dependency tracking during rendering (same principle as computed()). When a signal's value changes, Angular knows precisely which components consumed it - CD is targeted, not tree-wide.

```
signal.set(newValue)
  -> Angular's internal reactive graph marks only the consuming
     component(s) dirty
  -> CD scheduled (microtask) - only dirty components checked,
     not the whole tree
```

This is why Signals fundamentally change CD's cost model: Zone.js CD is O(entire tree) per trigger. Signal CD is O(components that actually depend on the changed value).

What "Running CD" Actually Does, Mechanically

```
For a component being checked:
1. Re-evaluate every template expression/binding ({{ }}, [prop],
   (event) context reads)
2. Compare new value to previously rendered value (reference
   equality for objects, value equality for primitives)
3. If different -> patch that specific DOM node/attribute (Ivy
   generates precise instructions per binding - not innerHTML
   replacement)
4. Recurse into child components (Zone.js era) or skip children
   with no dirty signals (Signals era)
```

This comparison step is why ExpressionChangedAfterItHasBeenCheckedError exists - dev mode runs a second verification pass after the real one to catch bindings that changed after being checked (e.g. mutating template-bound state inside ngAfterViewInit).

OnPush vs Default — Where They Fit In This Model

Default: component is checked on every CD pass, regardless of whether its own inputs changed. Cheapest to write, expensive to run at scale.

OnPush: component is skipped unless one of these fires - an @Input()/input() reference changes (=== check, not deep equality, which is why mutating an array in place doesn't trigger OnPush but reassigning does), an event originates from within that component's own template, a signal read inside its template changes, or markForCheck()/detectChanges() is called manually.

OnPush is really "opt this component out of the blind tree-walk, only re-check on explicit dirty signals." With Signals as the default model, OnPush essentially becomes automatic - signal-only components approximate OnPush behavior for free, since dirty-checking is inherently targeted.

When It Happens — Full Trigger List

Trigger	Zone.js era	Zoneless/S
Signal .set()/.update()	Triggers full tree CD (Zone catches it indirectly via effect scheduling)	Triggers ta
DOM event (click, input)	Zone intercepts, triggers CD	Same - eve
HTTP response	Zone intercepts, triggers CD	Only if resp
setTimeout/setInterval	Zone intercepts, triggers CD	Not tracked
Manual markForCheck()/detectChanges()	Forces a check	Same, still

This table is the reason zoneless apps require discipline: async work that doesn't touch a signal silently does nothing to the UI. In Zone.js era, everything async was an implicit trigger - a safety net that also caused the tree-walk cost.

Why Angular Moved This Direction

Zone.js CD cost scales with tree size regardless of what actually changed - every setTimeout anywhere in the app triggers a full walk. Signals let Angular know the exact dependency graph ahead of time, collapsing CD from "check everything, hope OnPush filters most of it" to "check only what's provably dirty." It's the same shift React made with fine-grained reactivity libraries (Solid, etc.) - dependency-tracked updates beat tree-walk-and-diff at scale.

Complete Flow — Bootstrap Through tick()

What CD literally is, mechanically: CD is not a listener or an event system - it's a function (ApplicationRef.tick()) that walks the component tree and runs each component's generated check function (Ivy compiles one per component from the template). That function reads every binding, compares to the last stored value, patches DOM if different. CD "running" = this function executing top to bottom.

When It Starts — Bootstrap Flow

```
bootstrapApplication(AppComponent)
  -> ApplicationRef created
  -> Zone.js (if not zoneless) wraps app in NgZone
  -> root component instantiated, first tick() forced manually
  -> from here, tick() gets re-invoked by triggers (see below)
```

First CD run happens right after bootstrap, unconditionally - even with nothing changed, Angular has to render initial state once.

The Trigger -> tick() Flow (Zone.js Era)

```
1. Async API called (click handler, setTimeout, HTTP, Promise.then)
2. Zone.js's monkey-patched version of that API runs
3. Zone.js wraps the actual callback execution and, AFTER it
   completes, calls NgZone's "onMicrotaskEmpty"
4. NgZone tells ApplicationRef: run tick()
5. ApplicationRef.tick() walks the ENTIRE component tree, root
   to leaf
```

Step 3 is the important mechanism: Zone.js doesn't trigger CD on every single async call - it batches. It waits until the microtask queue is empty (all synchronous+microtask work from that event finished), then fires ONE tick(). This is why multiple state changes inside one click handler only cause one DOM patch, not one per line.

The Tree Walk Itself — tick() Internals

```
tick()
  -> starts at root component
  -> for EACH component (regardless of OnPush, in Zone.js era it
     still visits):
      a. Check: is this component "dirty" and eligible to be
         checked?
         - Default strategy: always eligible
         - OnPush strategy: eligible only if flagged dirty (input
           ref changed, event originated here, or manual
           markForCheck)
      b. If eligible: run generated check function
         - Evaluate every binding expression in the template
         - Compare new value to cached value (reference equality)
         - If different -> call DOM update instruction (e.g.
           \u0275\u0275textInterpolate, \u0275\u0275property) -> real
           DOM mutated
      c. Recurse into child components, same process
  -> after full walk: dev mode runs SECOND pass (checkNoChanges)
     to catch late mutations -> throws
     ExpressionChangedAfterItHasBeenCheckedError if a binding
     changed between pass 1 and pass 2
```

Critical detail: even with OnPush, the tree walk itself still visits every component - OnPush just skips step (b) for components not flagged dirty. It doesn't prune the walk, it prunes the check. This is why OnPush helps but doesn't eliminate tree-walk cost entirely in Zone.js era.

The Signals-Era Flow

```
1. signal.set(newValue) called
2. Signal's internal version counter increments
3. Angular's reactive graph (built during LAST render, when the
   signal was READ inside a template) already knows exactly which
   component(s) read this signal
4. Those specific components get marked dirty directly - no tree
   walk needed to FIND them
5. Angular schedules a CD pass (coalesced into a microtask,
   similar batching principle to Zone.js's tick), but the walk
   that follows only touches dirty components + their ancestors
   (ancestors must be visited structurally to reach the dirty
   node, but non-dirty siblings are skipped entirely)
```

The dependency graph in step 3 is built via the same mechanism as computed() - when a template renders, every signal read is tracked ("this component consumed this signal"), so future writes know their exact blast radius.

Full Sequence — Concrete Zone.js Example

```
User clicks button
  -> (click) handler fires, patched by Zone.js
  -> this.count++ runs inside handler
  -> handler finishes, microtask queue empties
  -> Zone notifies NgZone -> NgZone calls ApplicationRef.tick()
  -> tick() walks tree from root
  -> reaches component with {{ count }} binding
  -> OnPush check: was count's containing component flagged dirty?
       (yes - event originated in its own template)
  -> generated check fn runs -> old value 4, new value 5 ->
     different
  -> \u0275\u0275textInterpolate1 instruction fires -> DOM text
     node updated
  -> walk continues to remaining components
  -> dev mode: checkNoChanges second pass verifies nothing changed
     since being checked
  -> tick() returns, browser paints
```

Core Rules to Remember

CD is a synchronization function, not an event system - the only real design question across both eras is how does Angular know which components need re-syncing, and does it have to walk the whole tree to find out, or does it already know. Zone.js era: walk everything, filter with OnPush. Signals era: know exactly what's dirty, walk only what's necessary to reach it.
